DTI201 • บทที่ 8

จาก React + Vite สู่ Express API

สร้าง API หนังสือ แล้วเชื่อมกลับไปยังแอปที่ทำไว้





6 กิจกรรม • จากคำขอแรกถึงการจัดการข้อผิดพลาด

หนังสือบทที่ 8 | React frontend → HTTP → Express backend

แผนที่บทเรียน

จากหน้าจอเดิม → API ที่ใช้งานได้

เส้นทางของข้อมูล ผู้ใช้กดปุ่มใน React แล้วติดตามคำขอไปจนเห็นผลบนหน้าจอ



ลำดับการเรียน: 5 ช่วง / 6 กิจกรรม

01	02	03	04	05
เริ่มจากแอปเดิม	**เส้นทางคำขอ**	**อ่านและตรวจค่า**	**เพิ่มหนังสือ**	**ข้อผิดพลาด**
คำ พื้น ฐาน และ เปิด เซิร์ฟเวอร์	route และ middleware	GET, POST และค้นหนังสือ	ฟอร์ม, 201 และซื่อซ้ำ	404, 500 และ requestId

เป้าหมาย: React แสดงหนังสือที่ Express ส่งกลับมา

กดปุ่ม “เพิ่มหนังสือ” แล้วเกิดอะไรขึ้น?

ในบทก่อน เราทำหน้ารายการหนังสือด้วย React และ Vite แล้ว ผู้ใช้พิมพ์ชื่อหนังสือในฟอร์มและกดปุ่มได้ แต่ถ้าเก็บข้อมูลไว้แค่ในหน้าจอ ข้อมูลในหน้าจออาจหายเมื่อโหลดหน้าใหม่

บทนี้เราจะทำโปรแกรมฝั่งเซิร์ฟเวอร์ให้รับข้อมูลจากหน้า React ตรวจข้อมูล แล้วส่งผลกลับมา มีทั้งหมด **6 กิจกรรม** เริ่มจากดูคำขอจริง ไปจนถึงเพิ่มหนังสือและตรวจข้อผิดพลาด



ตัวอย่างที่เห็นได้จริง

React ส่ง `POST /api/books` พร้อมชื่อหนังสือ เซิร์ฟเวอร์ตอบ `201` เมื่อเพิ่มสำเร็จ หรือ `422` พร้อมบอกว่าช่องใดกรอกผิด

ลองคิดหรือสังเกต: หน้าจอ React ที่ทำไว้แล้วมีหน้าที่อะไร และยังขาดส่วนใด?

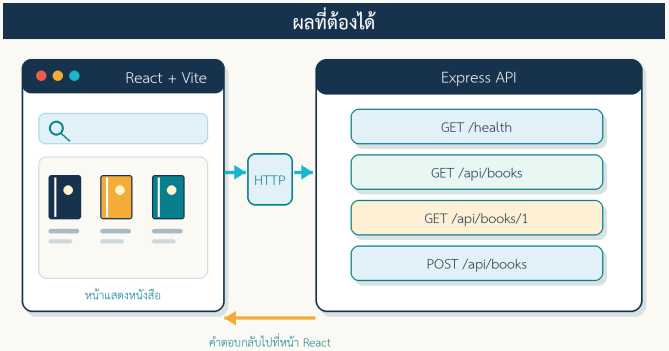
เชื่อมกับแอป React + Vite: ปุ่ม “เพิ่มหนังสือ” ใน React ส่งคำขอไปยัง Express แล้วรอผลกลับมา

อ่านหนังสือบทที่ 8 §8.1

บทนี้เราจะทำอะไรให้ใช้ได้จริง?

ผลลัพธ์สุดท้ายคือโปรแกรมเล็ก ๆ ที่ตอบคำขอเกี่ยวกับหนังสือได้: ตรวจว่าเซิร์ฟเวอร์ทำงานอยู่ แสดงรายการหนังสือ แสดงหนังสือหนึ่งเล่ม และรับหนังสือเล่มใหม่

ภาพด้านขวาคือผลที่ต้องการ: หน้า React แสดงหนังสือที่เพิ่งเพิ่ม ส่วน Express รับและตอบคำขอ เราจะทำ 6 กิจกรรม: 0 ดูคำตอบจริง, 1 แยก GET/POST, 2 แบ่งหน้า, 3 ค้นด้วยรหัส, 4 เพิ่มหนังสือ, 5 ตรวจข้อผิดพลาด



ตัวอย่างที่เห็นได้จริง

เมื่อจบคาบ ลองส่งคำขอ 4 แบบ: `GET /health`, `GET /api/books`, `GET /api/books/1`, `POST /api/books`

ลองคิดหรือสังเกต: ถ้าหน้า React ขอ `/api/books` คำตอบควรมีอะไรให้แสดง?

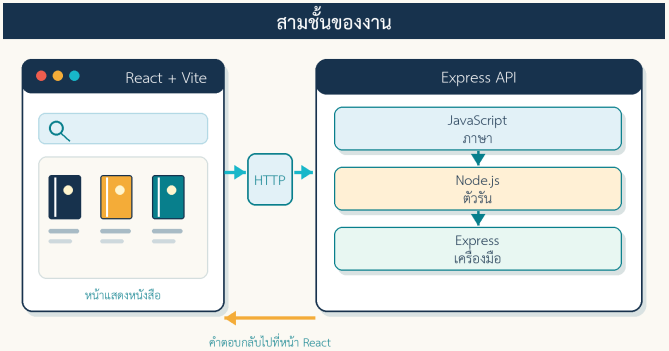
เชื่อมกับแอป React + Vite: หน้า React แสดงรายการจาก GET และแสดงหนังสือใหม่หลัง POST สำเร็จ

อ่านหนังสือบทที่ 8 §8.1, §8.10

JavaScript, Node.js และ Express ต่างกันอย่างไร?

JavaScript คือภาษาที่เราใช้เขียนคำสั่ง **Node.js** เป็นโปรแกรมที่รัน JavaScript นอกเบราว์เซอร์ ส่วน **Express** เป็นชุดเครื่องมือสำหรับเขียนเซิร์ฟเวอร์บน Node.js

ดังนั้น Express ไม่ใช่ภาษาใหม่ เราจะยังเขียน JavaScript แบบที่เคยใช้ แต่เพิ่มวิธีรับคำขอจากเครือข่าย



ตัวอย่างที่เห็นได้จริง

ไฟล์ `server.js` เป็น JavaScript; Node.js เปิดไฟล์นี้; Express ช่วยจับคู่ `GET /health` กับฟังก์ชันที่ตอบคำขอ

ลองคิดหรือสังเกต: สามชื่อนี้ตัวใดเป็นภาษา ตัวใดเป็นโปรแกรมรัน และตัวใดเป็นชุดเครื่องมือ?

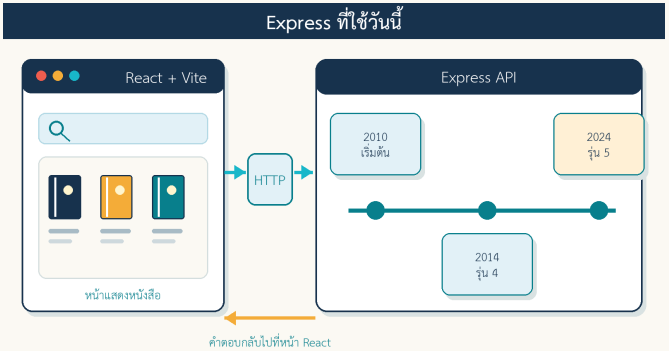
เชื่อมกับแอป React + Vite: React กับ Express ใช้ JavaScript เหมือนกัน แต่ทำงานคนละที่

อ่านหนังสือบทที่ 8 §8.3

รู้ที่มาสั้น ๆ ก่อนลงมือ

Express มีการใช้งานมาตั้งแต่ราวปี 2010 รุ่น 4 เริ่มใช้ในปี 2014 และรุ่น 5 ออกในปี 2024 เราใช้รุ่น 5 ในคาบนี้

สิ่งที่เปลี่ยนไปในรุ่น 5 ที่เกี่ยวกับบทเรียนคือ เมื่อฟังก์ชันแบบ **async** เกิดข้อผิดพลาด Express สามารถส่งต่อไปยังตัวจัดการข้อผิดพลาดได้



ตัวอย่างที่เห็นได้จริง

ดู ประกาศ Express 5 และ ตารางรุ่นของ Express แล้วสังเกตว่าตัวอย่างในคาบนี้ต้องใช้ Node.js รุ่นใหม่

ลองคิดหรือสังเกต: ประวัติช่วยให้รู้ว่าทำไมตัวอย่างในเอกสารเก่าบางแห่งจึงเขียนไม่เหมือนกัน

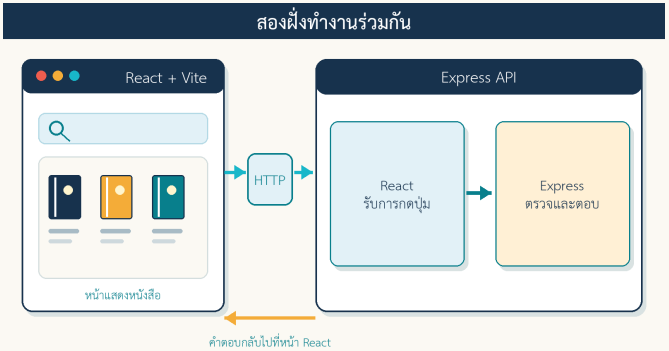
เชื่อมกับแอป React + Vite: หน้า React ที่นักศึกษาทำไว้จะคุยกับ Express รุ่น 5 ในบทนี้

อ่านหนังสือบทที่ 8 บทนำ

หน้า React กับเซิร์ฟเวอร์ทำงานคนละอย่าง

หน้า React ทำงานในเบราว์เซอร์: แสดงฟอร์ม รับการกดปุ่ม และบอกผู้ใช้ว่าเกิดอะไรขึ้น ส่วนเซิร์ฟเวอร์รับคำขอจากทุกทาง ไม่ว่าจะมาจาก React หรือโปรแกรมอื่น

ถ้า React ตรวจชื่อหนังสือว่าไม่ว่างเพื่อช่วยผู้ใช้ เซิร์ฟเวอร์ก็ยังต้องตรวจอีกครั้ง เพราะคนส่งคำขอโดยไม่ผ่านฟอร์มได้



ตัวอย่างที่เห็นได้จริง

ผู้ใช้กดเพิ่มหนังสือใน React หรือส่ง `curl` ตรงมาที่เซิร์ฟเวอร์ ผลการตรวจข้อมูลต้องเหมือนกัน

ลองคิดหรือสังเกต: ถ้าเราปิดการตรวจข้อมูลในเซิร์ฟเวอร์ คนส่ง `curl` จะทำอะไรได้?

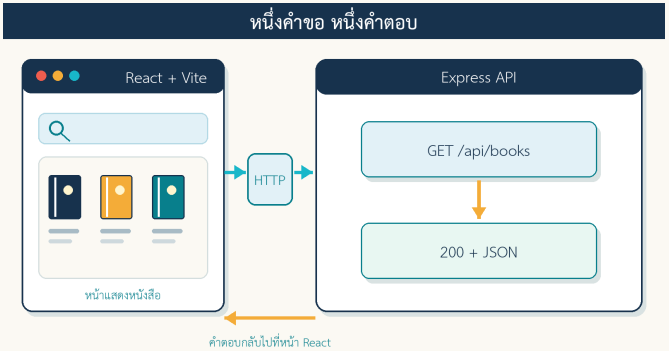
เชื่อมกับแอป React + Vite: React รับการกระทำของผู้ใช้ ส่วน Express ตรวจข้อมูลก่อนตอบ

อ่านหนังสือบทที่ 8 §8.1

คำขอและคำตอบคืออะไร?

เมื่อเบราว์เซอร์ต้องการข้อมูล จะส่ง **คำขอ** (request) ไปยังเซิร์ฟเวอร์ เซิร์ฟเวอร์ทำงานแล้วส่ง **คำตอบ** (response) กลับมา หนึ่งคำขอมีหนึ่งคำตอบที่เราต้องตรวจ

คำตอบมีรหัสสถานะ เช่น `200` และอาจมีข้อมูล JSON ด้วย การเห็นรายการบนจอจึงเป็นผลจากการส่งและรับข้อมูลผ่านเครือข่าย



ตัวอย่างที่เห็นได้จริง

`GET /api/books` เป็นคำขอ; `200` กับรายการหนังสือเป็นคำตอบ เปิดแท็บ Network ใน DevTools เพื่อดูทั้งสองด้าน

ลองคิดหรือสังเกต: ถ้าหน้าจอว่าง เราจะดูตรงไหนก่อนว่าเซิร์ฟเวอร์ตอบอะไร?

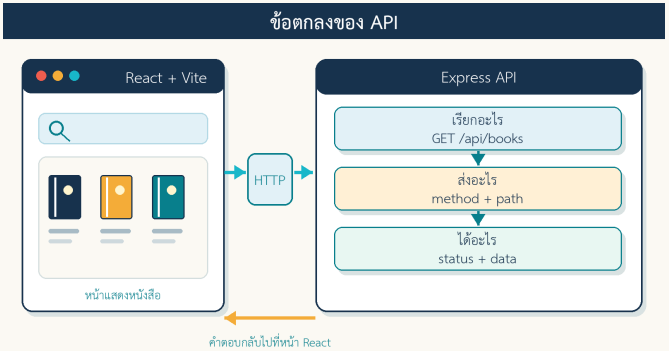
เชื่อมกับแอป React + Vite: React ส่ง request และอ่าน status กับ JSON ใน response

อ่านหนังสือบทที่ 8 §8.1

API คือข้อตกลงระหว่างสองโปรแกรม

API กำหนดว่าโปรแกรมหนึ่งจะขอให้อีกโปรแกรมทำอะไรได้ ต้องส่งข้อมูลแบบไหน และจะได้คำตอบแบบไหน หน้ารายการหนังสือไม่จำเป็นต้องรู้ว่าเซิร์ฟเวอร์เก็บข้อมูลอย่างไร

ในคาบนี้ “ทรัพยากร” คือหนังสือ เราใช้ที่อยู่ `/api/books` สำหรับรายการหนังสือ และ `/api/books/1` สำหรับหนังสือรหัส 1



ตัวอย่างที่เห็นได้จริง

React ขอ `GET /api/books`; เซิร์ฟเวอร์ส่ง `{data:[...]}` กลับมา React นำ `data` ไปแสดงเป็นการ์ด

ลองคิดหรือสังเกต: ถ้าเซิร์ฟเวอร์เปลี่ยนรูปคำตอบ หน้า React จะได้รับผลอย่างไร?

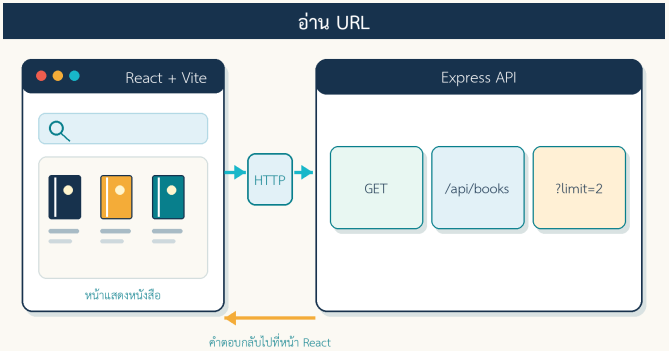
เชื่อมกับแอป React + Vite: ฟังก์ชัน bookApi.ts ของ React ต้องรู้ข้อตกลงของ API

อ่านหนังสือบทที่ 8 §8.1

อ่านที่อยู่ของคำขอให้เป็น

คำขอแต่ละครั้งมี **วิธีการ** เช่น GET, **เส้นทาง** เช่น `/api/books`, และอาจมี **ค่าที่ต่อท้าย URL** เช่น `?limit=2` วิธีการบอกว่าอยากทำอะไร เส้นทางบอกว่าทำกับข้อมูลชุดใด

GET ใช้ขออ่านข้อมูล ส่วน POST ใช้ส่งข้อมูลเพื่อสร้างรายการใหม่ คำว่า `limit` หลังเครื่องหมายคำถามเป็น query ไม่ใช่ชื่อเส้นทางใหม่



ตัวอย่างที่เห็นได้จริง

`GET /api/books?limit=2` หมายถึงขอรายการหนังสือ โดยขอไม่เกิน 2 รายการ

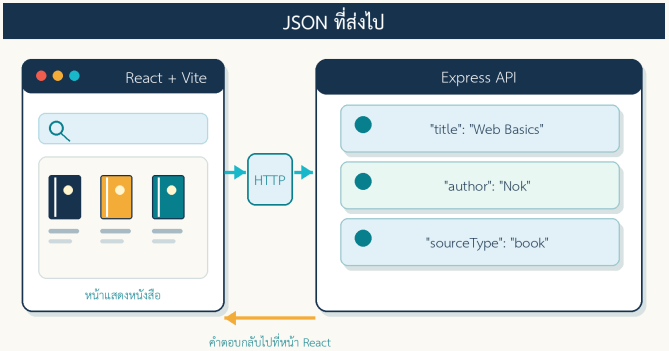
ลองคิดหรือสังเกต: วงวิธีการ เส้นทาง และ query ในตัวอย่างให้ครบ

เชื่อมกับแอป React + Vite: React เลือก method, path และ query ก่อนเรียก fetch

อ่านหนังสือบทที่ 8 §8.5

JSON คือข้อความที่ใช้ส่งข้อมูล

JSON เป็นรูปแบบข้อความที่เก็บคู่ชื่อกับค่า เช่น `title` คู่กับชื่อหนังสือ เราใช้ JSON ในคำขอ POST และในคำตอบจาก API ต้องเขียนชื่อฟิลด์ในเครื่องหมายคำพูด และใช้เครื่องหมายจุลภาคคั่นฟิลด์ ถ้าพิมพ์ JSON ผิดรูป เซิร์ฟเวอร์จะอ่านไม่ออกก่อนถึงขั้นตรวจชื่อหนังสือ



ตัวอย่างที่เห็นได้จริง

`{"title":"Web Basics","author":"Nok"}` เป็น JSON ที่อ่านได้ แต่ `{"title":}` อ่านไม่ได้

ลองคิดหรือสังเกต: “อ่าน JSON ไม่ได้” กับ “ชื่อหนังสือสั้นเกิน” เป็นความผิดพลาดขั้นเดียวกันหรือไม่?

เชื่อมกับแอป React + Vite: React ส่ง JSON จากฟอร์ม; Express ต้องอ่าน JSON ก่อนใช้

อ่านหนังสือบทที่ 8 §8.3, §8.6

ไฟล์เดิมกับไฟล์ใหม่อยู่ตรงไหน?

แอปเดิมใช้ `App.tsx` สำหรับหน้าจอ, `bookApi.ts` สำหรับคำสั่ง `fetch`, และ `package.json` สำหรับคำสั่งของ Vite

ฝั่งเซิร์ฟเวอร์ใน lab นี้ใช้ `app.js` สร้าง Express app, `server.js` เปิดพอร์ต, `books.js` เก็บหนังสือตัวอย่าง และ `package.json` ระบุ Express กับคำสั่ง `npm start`



ตัวอย่างที่เห็นได้จริง

`.tsx` เขียน UI ที่มี JSX; `.ts` เขียน TypeScript; `.js` เป็น JavaScript ที่ Node.js รัน; `.json` เก็บข้อมูลหรือการตั้งค่า

ลองคิดหรือสังเกต: ชี้ไฟล์ที่ต้องแก้เมื่อจะเพิ่มเส้นทาง `GET /api/books/1`

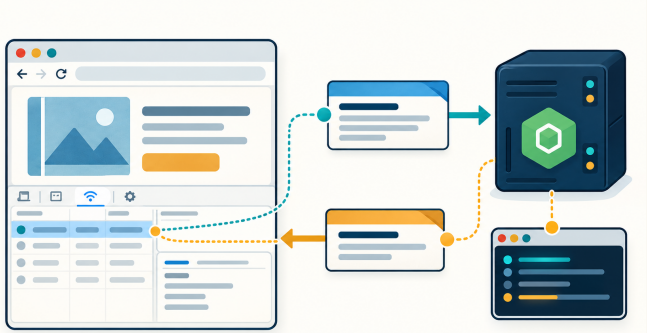
เชื่อมกับแอป React + Vite: App.tsx แสดงหน้า; bookApi.ts ส่งคำขอ; app.js รับคำขอ

อ่านหนังสือบทที่ 8 §8.3

เปิดเซิร์ฟเวอร์แล้วตรวจอย่างไร?

เปิด terminal ในโฟลเดอร์ `examples/09-express-book-api` แล้วรัน `npm ci` เพื่อติดตั้งแพ็กเกจตาม lockfile จากนั้นรัน `npm start` เพื่อให้เซิร์ฟเวอร์รอคำขอที่พอร์ต 3001

เปิด terminal อีกหน้าต่างไว้ทดสอบ อย่าสับสนกับ Vite ที่มักเปิดหน้า React ที่พอร์ต 5173: สองโปรแกรมทำงานคนละพอร์ต



ตัวอย่างที่เห็นได้จริง

รัน `curl -i http://localhost:3001/health` ควรเห็น `200` และ JSON ที่มี `"status":"ok"`

ลองคิดหรือสังเกต: ถ้า curl ต่อพอร์ต 3001 ไม่ได้ ให้ดู terminal ของเซิร์ฟเวอร์ก่อน

เชื่อมกับแอป React + Vite: ตรวจ API ด้วย curl ก่อนต่อกลับไปยัง React ในเบราว์เซอร์

อ่านหนังสือบทที่ 8 §8.3

ช่วงที่ 2 / 5

เส้นทางของคำขอ

จาก React มาถึง Express แล้วคำขอผ่านอะไรบ้าง?

ในช่วงนี้เราจะทำอะไร

01
Route เลือกฟังก์ชันจาก method และ path

02
Middleware เตรียมข้อมูลหรือส่งต่อ

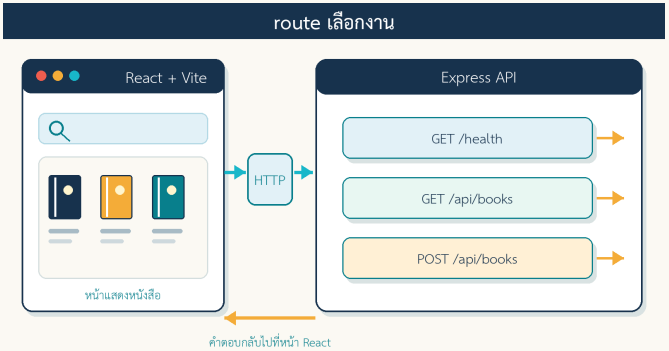
03
กิจกรรม 0: ทำนายผลแล้วตรวจด้วย curl

React + Vite → HTTP → Express → status + JSON → React

Express รู้ได้อย่างไรว่าต้องตอบอะไร?

เราเขียน **route** เพื่อบอก Express ว่า เมื่อมีวิธีการและเส้นทางตรงกับที่กำหนด ให้เรียกฟังก์ชันใด ฟังก์ชันนั้นอ่านคำขอ แล้วส่งคำตอบ

คำว่า `req` มาจาก request; `res` มาจาก response. เราจะใช้ชื่อนี้ซ้ำในตัวอย่าง เพื่อให้ตามคำขอในโค้ดได้ง่าย



ตัวอย่างที่เห็นได้จริง

`app.get('/health', (req, res) => { ... })` จับคำขอ GET ที่ `/health`; `res.json(...)` ส่ง JSON กลับ

ลองคิดหรือสังเกต: ถ้าส่ง POST ไปที่ `/health` จะใช้ route GET นี้หรือไม่?

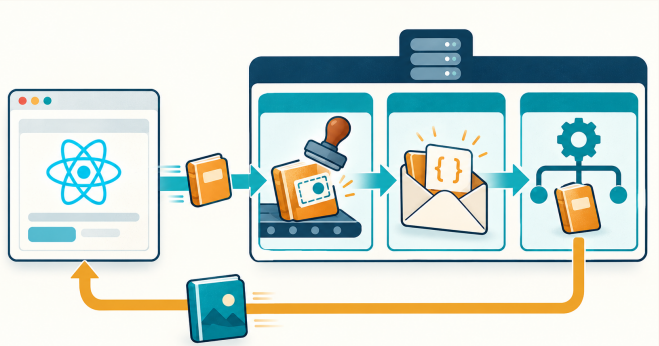
เชื่อมกับแอป React + Vite: เมื่อ React เรียก fetch เส้นทางและ method พาไปยัง route ที่ตรงกัน

อ่านหนังสือบทที่ 8 §8.5

Middleware คือขั้นตอนระหว่างทาง

ก่อนคำขอจะถึง route เราอาจให้ฟังก์ชันอื่นทำงานก่อน ฟังก์ชันเหล่านี้เรียกว่า **middleware** ใน starter มีขั้นตอนสร้างรหัสคำขอและขั้นตอนอ่าน JSON

Express ทำตามลำดับที่ลงทะเบียนไว้ ถ้าข้อมูล JSON ผิดรูป ขั้นตอนอ่าน JSON อาจหยุดคำขอก่อนถึง route



ตัวอย่างที่เห็นได้จริง

คำขอ POST เดินผ่าน “สร้างรหัสคำขอ” แล้ว “อ่าน JSON” ก่อนถึง “เพิ่มหนังสือ”

ลองคิดหรือสังเกต: เปิด `app.js` แล้วขีดเส้นใต้ `app.use(...)` สองจุดแรก

เชื่อมกับแอป React + Vite: คำขอจาก React ผ่าน middleware ก่อนถึง route

อ่านหนังสือบทที่ 8 §8.4

Middleware ต้องส่งต่อหรือส่งคำตอบ

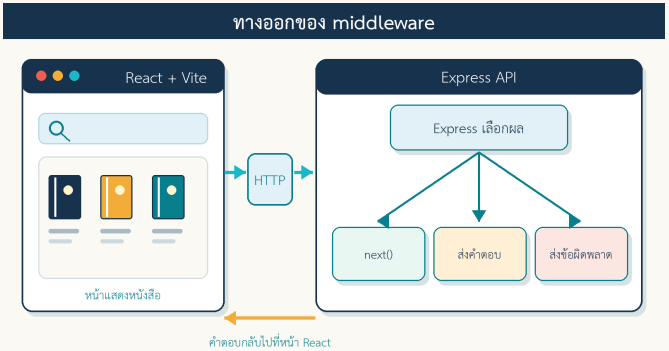
เมื่อ middleware ทำงานเสร็จ ต้องเลือกว่าจะเรียก `next()` ให้ขั้นถัดไปทำงาน หรือส่งคำตอบกลับทันที หากเกิดข้อผิดพลาดก็ต้องส่งต่อไปยังส่วนที่จัดการข้อผิดพลาด

ถ้าไม่ทำอย่างใดอย่างหนึ่ง เบราว์เซอร์จะรอคำตอบไปเรื่อย ๆ เราจึงตรวจเส้นทางการทำงานทุกครั้งที่เพิ่ม middleware

ตัวอย่างที่เห็นได้จริง

middleware สร้าง `requestId` แล้วเรียก `next()`; route ของ `/health` ใช้ `res.json(...)` จบคำขอ

ลองคิดหรือสังเกต: ใน starter ฟังก์ชันใดเรียก `next()` และฟังก์ชันใดตอบกลับเอง?



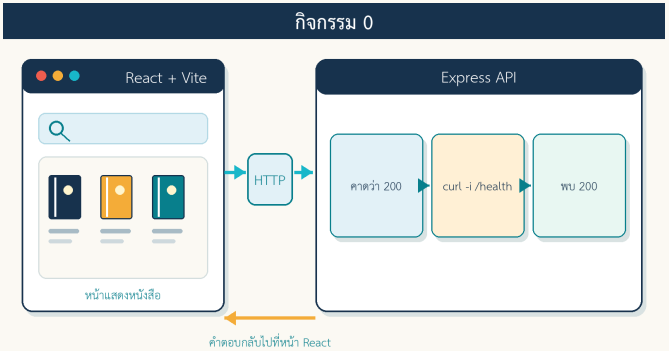
เชื่อมกับแอป React + Vite: ถ้า middleware ไม่ส่งต่อหรือไม่ตอบ React จะรออยู่บนหน้าจอ

อ่านหนังสือบทที่ 8 §8.4

กิจกรรม 0: ทำนายก่อนดูผล

เริ่มจากเส้นทางง่ายที่สุด: `/health` ใช้ตรวจว่าเซิร์ฟเวอร์รับคำขอได้ ก่อนกด Enter ให้เขียนว่าคิดว่าจะได้รหัสสถานะเท่าไรและมีข้อมูลอะไรกลับมา

จากนั้นลองเส้นทางที่ไม่ได้สร้างไว้ เช่น `/not-there` จดทั้งรหัสสถานะและคำตอบ แล้วกลับไปชี้บรรทัดใน `app.js` ที่ทำให้เกิดผลนั้น



ตัวอย่างที่เห็นได้จริง

ลอง `curl -i http://localhost:3001/health` และ `curl -i http://localhost:3001/not-there`

ลองคิดหรือสังเกต: ดูส่วนหัว `X-Request-Id` ด้วย รหัสนี้เหมือนกันระหว่างสองคำขอหรือไม่?

เชื่อมกับแอป React + Vite: เริ่มจาก curl เพื่อแยกปัญหา API ออกจากปัญหาหน้า React

อ่านหนังสือบทที่ 8 §8.4

ช่วงที่ 3 / 5

อ่านข้อมูลและตรวจค่าที่ส่งมา

ระบุให้ได้ว่าค่าอยู่ตรงไหน ก่อนนำไปใช้จริง

ในช่วงนี้เราจะทำอะไร

01

GET อ่านรายการ; POST รับข้อมูลใหม่

02

path, query และ body มีหน้าที่ต่างกัน

03

กิจกรรม 1–3: แบ่งหน้าและค้นด้วยรหัส

GET กับ POST ใช้เส้นทางเดียวกันได้

`GET /api/books` หมายถึงขออ่านรายการ ส่วน `POST /api/books` หมายถึงส่งข้อมูลเพื่อเพิ่มหนังสือ แม้เส้นทางเหมือนกัน แต่เป็นคนละคำสั่ง เพราะวิธีการ HTTP ต่างกัน

ลองนึกถึงหน้า React เดิม: เมื่อเปิดหน้าอาจใช้ GET; เมื่อกดปุ่ม “เพิ่ม” จะใช้ POST



ตัวอย่างที่เห็นได้จริง

`app.get('/api/books', ...)` และ `app.post('/api/books', ...)` เป็น route สองอัน

ลองคิดหรือสังเกต: กิจกรรม 1: รัน GET แล้วลอง POST ใน starter ทำไมผลจึงต่างกัน?

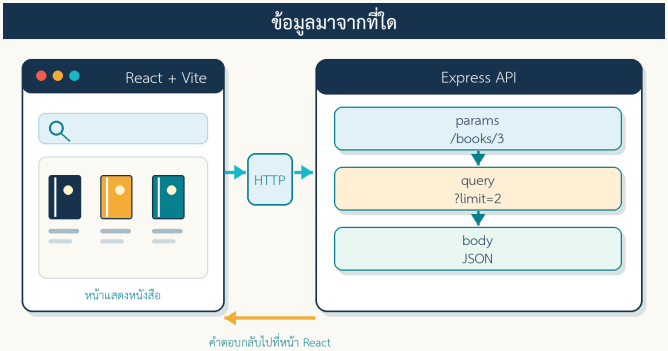
เชื่อมกับแอป React + Vite: React ใช้ GET เมื่อเปิดรายการ และ POST เมื่อกดเพิ่มหนังสือ

อ่านหนังสือบทที่ 8 §8.5

ข้อมูลที่ส่งมาอยู่ตรงไหนบ้าง?

params อยู่ในเส้นทาง เช่นเลข 3 ใน `/api/books/3`; **query** อยู่หลัง `?` เช่น `limit=2`; **body** เป็นข้อมูลที่ส่งมาในคำขอ เช่น JSON ของหนังสือใหม่

แต่ละตำแหน่งมีหน้าที่ต่างกัน เราจึงตรวจแยกกัน ไม่ควรเดาว่าตัวเลขจาก URL เป็น number แล้ว



ตัวอย่างที่เห็นได้จริง

`req.params.id`, `req.query.limit`, `req.body.title` อ่านข้อมูลจากคนละตำแหน่ง

ลองคิดหรือสังเกต: ในคำขอ `POST /api/books?draft=1` คำว่า draft อยู่ที่ใด?

เชื่อมกับแอป React + Vite: React ใส่รหัสใน path, ตัวเลือกใน query และข้อมูลใหม่ใน body

อ่านหนังสือบทที่ 8 §8.6

ตรวจข้อมูลก่อนนำไปใช้

ข้อมูลจากผู้ใช้ต้องผ่านการตรวจที่เซิร์ฟเวอร์ เพราะอาจพิมพ์ผิด ส่งผิดชนิด หรือส่งฟิลด์ที่เราไม่ต้องการ การตรวจนี้เรียกว่า **validation**

ตัวอย่างเช่น ก่อนใช้ `limit` ตัดรายการหนังสือ ต้องตรวจว่าเป็นจำนวนเต็มที่อยู่ในช่วง 1 ถึง 100 หากไม่ผ่าน ให้บอกว่าฟิลด์ใดผิด



ตัวอย่างที่เห็นได้จริง

`limit=2` ใช้ได้ แต่ `limit=abc` หรือ `limit=0` ต้องได้ `422` พร้อมคำอธิบาย

ลองคิดหรือสังเกต: ถ้าไม่ตรวจ `limit=0` ผู้ใช้จะเข้าใจผิดว่าไม่มีหนังสือหรือไม่?

เชื่อมกับแอป React + Vite: แม้ฟอร์ม React ตรวจแล้ว Express ก็ต้องตรวจข้อมูลซ้ำ

อ่านหนังสือบทที่ 8 §8.6

ตัวเลขใน URL เริ่มต้นเป็นข้อความ

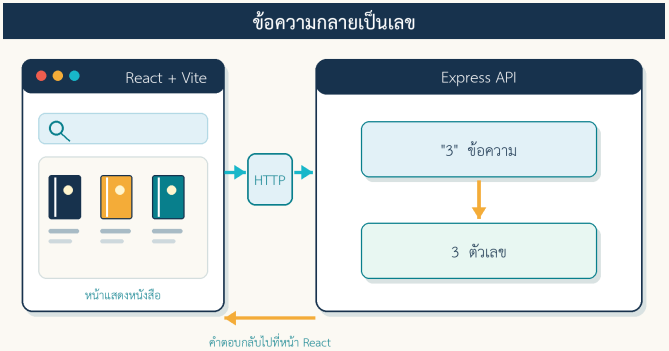
เมื่ออ่าน `/api/books/3`, ค่า `req.params.id` เริ่มเป็นข้อความ `"3"` ก่อนนำไปใช้เป็นรหัสเลข เราต้องตรวจรูปแบบแล้วค่อยแปลงเป็น `3`

การใช้ `Number(...)` อย่างเดียวไม่พอ: ข้อความว่างกลายเป็น 0 ได้ และข้อความที่มีจุดทศนิยมอาจไม่ใช่รหัสที่เรายอมรับ

ตัวอย่างที่เห็นได้จริง

`Number("3")` ได้ 3 แต่ `Number("")` ได้ 0; ทั้งสองกรณีต้องคิดว่าอนุญาตหรือไม่

ลองคิดหรือสังเกต: ก่อนเรียก `store.find(id)` เราควรตรวจเงื่อนไขใดบ้าง?



เชื่อมกับแอป React + Vite: รหัสที่ React ต่อใน URL ไปถึง Express ในรูปข้อความ

อ่านหนังสือบทที่ 8 §8.6

แบ่งหน้า: limit และ offset

`limit` คือจำนวนรายการสูงสุดที่ขอ ส่วน `offset` คือจำนวนรายการที่ข้ามไปก่อนเริ่มแสดง ข้อมูลตัวอย่างมีหนังสือสองเล่มที่รหัส 1 และ 2

ถ้าขอ `limit=1&offset=1` ระบบข้ามเล่มแรก แล้วส่งเล่มที่สองกลับมา ข้อมูลหนังสือยังอยู่ครบในเซิร์ฟเวอร์ เพียงแต่คำตอบนี้เลือกส่งบางส่วน



ตัวอย่างที่เห็นได้จริง

`GET /api/books?limit=1&offset=1` ควรได้หนังสือรหัส 2 เพียงเล่มเดียว

ลองคิดหรือสังเกต: ลองเปลี่ยน offset เป็น 0 แล้วเปรียบเทียบผล

เชื่อมกับแอป React + Vite: React ขอหน้าที่ต้องการ; Express ส่งเฉพาะหนังสือส่วนนั้นกลับ

อ่านหนังสือบทที่ 8 §8.6

กิจกรรม 2: ตรวจ query ที่ใช้แบ่งหน้า

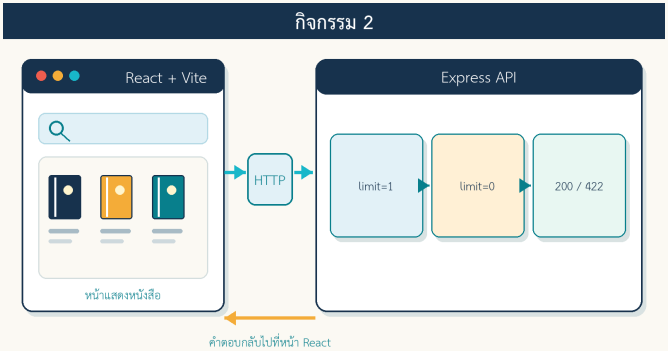
ใน starter ค่า `limit` และ `offset` ยังเขียนตายตัว ให้แก้ route รายการหนังสือเพื่ออ่าน query ตรวจรูปแบบ ตรวจช่วงค่า แล้วจึงส่งให้ store

ทำนายก่อนรันแต่ละคำขอ แล้วจดรหัสสถานะและรายการที่ได้ในใบงาน กรณีผิดควรได้ 422 พร้อมชื่อฟิลด์ที่ผิด

ตัวอย่างที่เห็นได้จริง

ลอง `?limit=1&offset=1`, `?limit=0`, `?offset=-1` ทีละคำขอ

ลองคิดหรือสังเกต: ผลของ `limit=0` หลังเพิ่ม validation ต่างจากก่อนแก้อย่างไร?



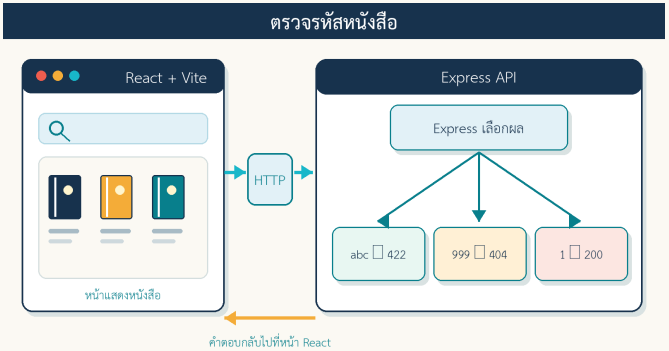
เชื่อมกับแอป React + Vite: ถ้า query ผิด React ใช้คำตอบ 422 บอกผู้ใช้ให้แก้

อ่านหนังสือบทที่ 8 §8.6

รหัสผิดรูป กับ รหัสที่ไม่มีข้อมูล

ถ้า URL เป็น `/api/books/abc` ค่า `abc` ใช้เป็นรหัสเลขไม่ได้ จึงตอบ `422` แต่ถ้าเป็น `/api/books/999` รหัสมีรูปแบบถูกต้อง เพียงแต่ไม่มีหนังสือเล่มนั้น จึงตอบ `404`

สองกรณีนี้ต้องแยก เพราะผู้ใช้และโปรแกรมที่เรียก API ควรแก้ปัญหาคนละแบบ



ตัวอย่างที่เห็นได้จริง

`/api/books/1` ได้ `200`; `/api/books/abc` ได้ `422`; `/api/books/999` ได้ `404`

ลองคิดหรือสังเกต: คำตอบกรณีใดควรมี `errors.id` และเพราะอะไร?

เชื่อมกับแอป React + Vite: React แยกข้อความ “รหัสผิด” ออกจาก “ไม่พบหนังสือ” ได้จาก status

อ่านหนังสือบทที่ 8 §8.6

กิจกรรม 3: ขอหนังสือหนึ่งเล่ม

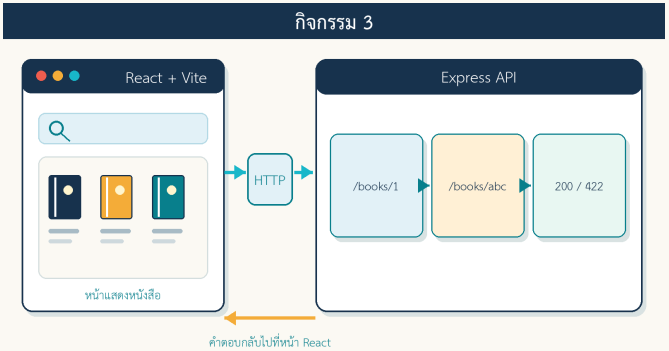
แทนคำตอบ `501` ใน starter ด้วยการตรวจรหัสแล้วค้นจาก store ให้ได้สามผลลัพธ์: พบหนังสือ รหัสผิดรูป และรหัสถูกแต่ไม่พบ

เมื่อแก้แล้ว ให้ส่งคำขอทั้งสามแบบจาก terminal และเปิด `app.js` ชี้จุดที่ตัดสินแต่ละผล

ตัวอย่างที่เห็นได้จริง

ลอง `curl -i .../api/books/1`, `.../abc`, และ `.../999`

ลองคิดหรือสังเกต: ถ้า route มีอยู่แต่หนังสือไม่พบ ต่างจาก `/not-there` อย่างไร?



เชื่อมกับแอป React + Vite: ตรวจ route ด้วย curl ก่อนให้ React เปิดหน้ารายละเอียดหนังสือ

อ่านหนังสือบทที่ 8 §8.6

ช่วงที่ 4 / 5

เพิ่มหนังสือให้สำเร็จ

ส่งข้อมูลจากฟอร์ม ตรวจข้อมูล และตอบผลให้ React แสดงต่อ

ในช่วงนี้เราจะทำอะไร

01
ฟอร์มส่งข้อมูลหนังสือเป็น JSON

02
สำเร็จตอบ 201 และ Location

03
กิจกรรม 4: เพิ่มจริงและลองกรณีชื่อซ้ำ

React + Vite → HTTP → Express → status + JSON → React

POST ต้องส่งข้อมูลอะไรบ้าง?

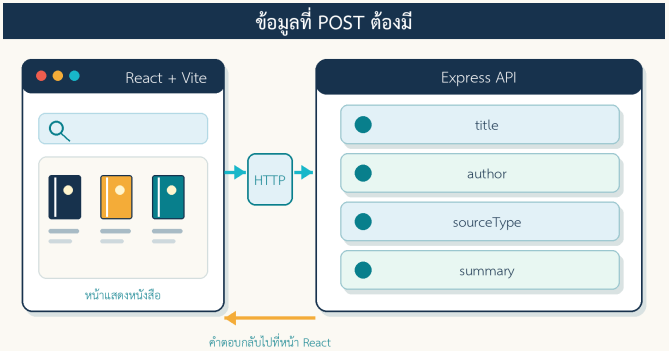
การเพิ่มหนังสือใช้ JSON ที่มีสี่ช่อง: `title`, `author`, `sourceType`, `summary` เซิร์ฟเวอร์ตัดช่องว่างหัวท้ายก่อนตรวจ และไม่รับฟิลด์ที่ไม่ได้ประกาศไว้

ชื่อหนังสือต้องยาวอย่างน้อย 3 ตัวอักษร ผู้แต่งและสรุปต้องไม่ว่าง ประเภทต้องเป็น `book`, `documentation` หรือ `course`

ตัวอย่างที่เห็นได้จริง

`{"title":"Node Practice","author":"Mali"}` และเติม `sourceType` กับ `summary` ตาม lab

ลองคิดหรือสังเกต: ถ้าเพิ่ม `"isAdmin":true` เข้าไป เซิร์ฟเวอร์ควรทำอย่างไร?



เชื่อมกับแอป React + Vite: ฟอร์ม React ส่ง title, author, sourceType และ summary ให้ Express

อ่านหนังสือบทที่ 8 §8.6

เมื่อเพิ่มสำเร็จ จะเห็นอะไรในคำตอบ?

ถ้าข้อมูลผ่านการตรวจและบันทึกสำเร็จ เซิร์ฟเวอร์ตอบ `201 Created`. ส่วนหัว `Location` บอกที่อยู่ของหนังสือเล่มใหม่ และ JSON ในคำตอบบอกรายละเอียดของเล่มนั้น

เราใช้ Location ตรวจซ้ำได้ทันที: ส่ง GET ไปยังที่อยู่นั้น แล้วดูว่าหนังสือใหม่อยู่จริงหรือไม่



ตัวอย่างที่เห็นได้จริง

POST สำเร็จได้ `Location: /api/books/3`; จากนั้น `GET /api/books/3` ควรได้หนังสือที่เพิ่งเพิ่ม

ลองคิดหรือสังเกต: ใน DevTools Network หรือ `curl -i` หา Location ได้ตรงไหน?

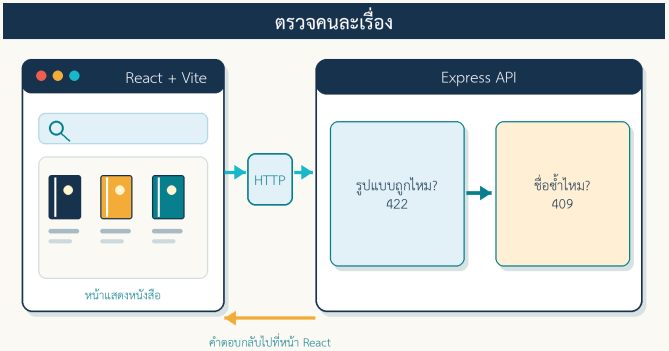
เชื่อมกับแอป React + Vite: React ใช้ผล 201 และ Location แสดงหนังสือเล่มที่เพิ่งเพิ่ม

อ่านหนังสือบทที่ 8 §8.5–8.7

ข้อมูลถูกตามรูปแบบ แต่ยังเพิ่มไม่ได้

การตรวจรูปแบบถามว่า “ข้อมูลเป็นข้อความที่ยาวพอหรือไม่” แต่กติกาของงานถามว่า “อนุญาตให้เพิ่มหนังสือนี้หรือไม่” ในตัวอย่างนี้ ชื่อหนังสือห้ามซ้ำ แม้ข้อมูลทุกช่องจะกรอกถูก

เราแยกสองเรื่องนี้เพื่อให้แก้โค้ดได้ตรงจุด และเมื่อเปลี่ยนที่เก็บข้อมูลในบทถัดไป กติกาชื่อซ้ำยังอยู่



ตัวอย่างที่เห็นได้จริง

ส่งหนังสือชื่อ `Node Practice` สองครั้ง ครั้งแรกได้ `201`; ครั้งที่สองได้ `409`

ลองคิดหรือสังเกต: ชื่อ `node practice` ตัวพิมพ์เล็กควรถือว่าซ้ำหรือไม่?

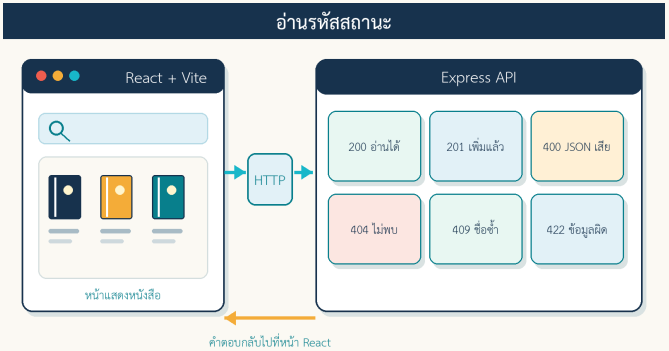
เชื่อมกับแอป React + Vite: เมื่อชื่อซ้ำ React ควรเก็บข้อมูลในฟอร์มไว้ให้ผู้ใช้แก้

อ่านหนังสือบทที่ 8 §8.7

อ่านรหัสสถานะให้เป็น

รหัสสถานะเป็นตัวเลขสั้น ๆ ที่บอกผลของคำขอ: `200` อ่านสำเร็จ, `201` สร้างสำเร็จ, `400` อ่าน JSON ไม่ได้, `404` ไม่พบ, `409` ชื่อซ้ำ, `422` ข้อมูลไม่ผ่านเงื่อนไข

อย่าจำตัวเลขอย่างเดียว ให้ถามก่อนว่าเกิดอะไรขึ้น แล้วจึงเลือกรหัสที่ตรงเหตุการณ์



ตัวอย่างที่เห็นได้จริง

`POST` ที่ส่ง JSON ผิดรูปได้ `400`; ส่ง JSON ถูกแต่ชื่อสั้นเกินได้ `422`

ลองคิดหรือสังเกต: สองคำขอนี้ผิดคนละขั้นตอนใดของเส้นทางคำขอ?

เชื่อมกับแอป React + Vite: React อ่าน status ก่อนตัดสินใจแสดงรายการหรือข้อความผิดพลาด

อ่านหนังสือบทที่ 8 §8.5, §8.8

กิจกรรม 4: เพิ่มหนังสือจริง

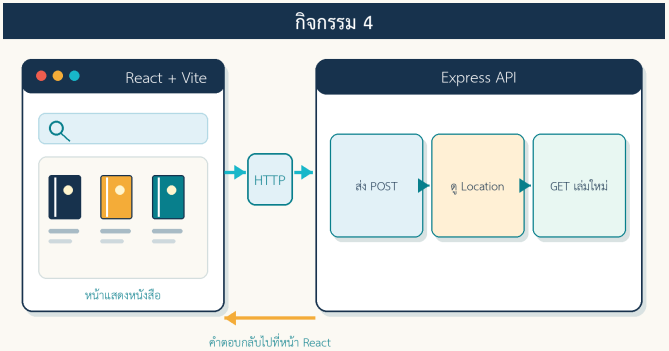
แก้ route POST ใน starter ให้ตรวจข้อมูลทั้งสี่ช่อง ตรวจชื่อซ้ำ บันทึกหนังสือ แล้วส่ง `201` กับ Location เมื่อสำเร็จ

ทดสอบอย่างน้อยสามกรณี: ข้อมูลถูก ข้อมูลผิดหลายช่อง และชื่อซ้ำ จดคำขอที่ส่งกับผลที่เห็นจริง อย่าเพียงอ่านโค้ดแล้วเดา

ตัวอย่างที่เห็นได้จริง

ส่ง JSON ที่ถูกต้องหนึ่งครั้ง ตรวจ Location แล้วส่งซ้ำอีกครั้งเพื่อดู `409`

ลองคิดหรือสังเกต: ถ้าผู้ใช้ส่ง `title` ที่มีแต่ช่องว่าง จะผ่านหรือไม่?



เชื่อมกับแอป React + Vite: หลัง POST สำเร็จ React เพิ่มหนังสือใหม่ลงรายการที่เห็น

อ่านหนังสือบทที่ 8 §8.10

ช่วงที่ 5 / 5

รับมือข้อผิดพลาด

เมื่อคำขอไม่สำเร็จ ผู้ใช้และผู้พัฒนาควรรู้อะไร?

ในช่วงนี้เราจะทำอะไร

01
แยก 404, 422 และ 500 จากกัน

02
ติดตามงาน async ด้วย requestId

03
กิจกรรม 5: ส่ง JSON ผิดรูปและตรวจหลักฐาน

ไม่พบข้อมูล กับ โปรแกรมทำงานผิดพลาด

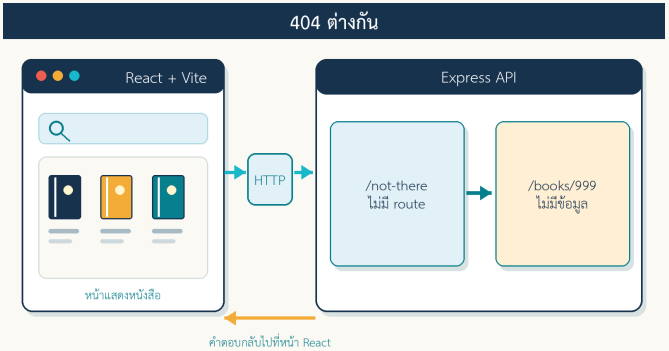
`/not-there` ไม่มี route รองรับ เราจึงส่งคำตอบ `404` จากส่วนท้ายของรายการ route ส่วน `/api/books/999` มี route แล้ว แต่ค้นหาหนังสือไม่พบ

ข้อผิดพลาดที่โปรแกรมทำงานพลาด เช่น store โยนข้อผิดพลาด ควรผ่านตัวจัดการข้อผิดพลาด และไม่ส่งรายละเอียดภายในกลับไป

ตัวอย่างที่เห็นได้จริง

`/not-there` ตอบ `Endpoint not found`; `/api/books/999` ตอบ `Book not found`

ลองคิดหรือสังเกต: ดูค่า `title` ในสองคำตอบ แล้วอธิบายว่าต่างกันตรงไหน



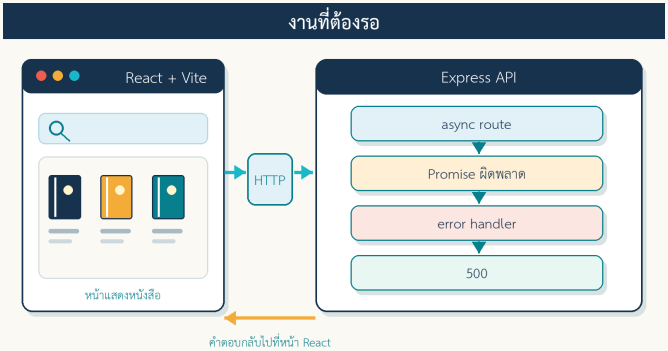
เชื่อมกับแอป React + Vite: React แสดงข้อความ 404 ให้ตรงกับกรณีที่ API รายงาน

อ่านหนังสือบทที่ 8 §8.8

งาน async ผิดพลาดแล้วไปไหน?

บางงานต้องรอ เช่น อ่านข้อมูลจากฐานข้อมูล จึงเขียน route แบบ `async`. ถ้า Promise ที่ route ส่งกลับเกิดข้อผิดพลาด Express 5 ส่งข้อผิดพลาดนั้นไปยัง error handler

error handler อยู่หลัง routes และรับ `error, req, res, next`. คำตอบ `500` ควรบอกว่าระบบมีปัญหา แต่ไม่เผยรหัสผ่านหรือข้อความภายใน



ตัวอย่างที่เห็นได้จริง

`async` route เรียก store; store ล้มเหลว; client ได้ `500 Internal server error`

ลองคิดหรือสังเกต: ถ้าเห็น stack trace ในคำตอบที่ส่งให้ผู้ใช้ ควรแก้ส่วนใด?

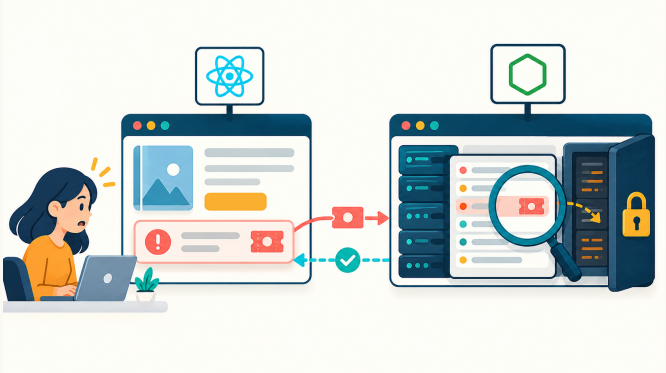
เชื่อมกับแอป React + Vite: ถ้าเซิร์ฟเวอร์ล้มเหลว React แสดงข้อผิดพลาดโดยไม่เผยข้อมูลภายใน

อ่านหนังสือบทที่ 8 §8.8

รหัสคำขอช่วยค้นหาสาเหตุ

แต่ละคำขอมี `requestId` ของตัวเอง เซิร์ฟเวอร์ใส่รหัสนี้ในส่วนหัว `X-Request-Id` และในคำตอบข้อผิดพลาด ผู้ใช้จึงแจ้งรหัสนี้ให้ผู้ดูแลค้นบันทึกของคำขอนั้นได้

บันทึกควรมีวิธีการ เส้นทาง รหัสสถานะ และเวลา ไม่ควรเก็บรหัสผ่าน token หรือ JSON body ทั้งก้อน



ตัวอย่างที่เห็นได้จริง

ส่งคำขอผิดหนึ่งครั้ง แล้วเทียบ `X-Request-Id` ใน header กับ `requestId` ใน JSON body

ลองคิดหรือสังเกต: สองค่านี้ควรตรงกันหรือไม่? ถ้าไม่ตรง จะตามปัญหาอย่างไร?

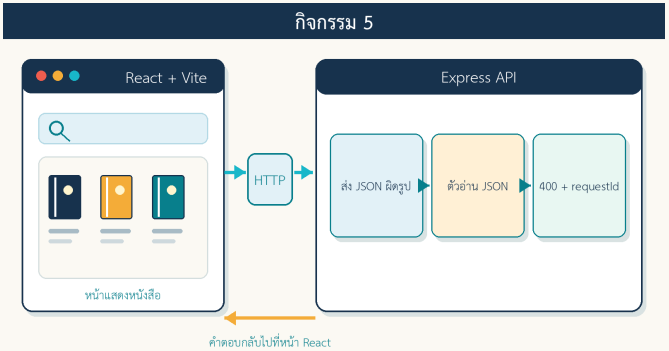
เชื่อมกับแอป React + Vite: React แสดง requestId ให้ผู้ใช้แจ้งเมื่อพบปัญหา

อ่านหนังสือบทที่ 8 §8.9

กิจกรรม 5: ลองส่ง JSON ผิดรูป

ส่ง POST ที่มี JSON ไม่ครบ เพื่อดูว่าโปรแกรมอ่านข้อมูลผิดก่อนถึง route เพิ่มหนังสือ ให้ error handler ตอบ `400` ในรูปแบบเดียวกับข้อผิดพลาดอื่น และมี request ID

ทำนายก่อนว่า route POST จะทำงานหรือไม่ แล้วดูผลจาก `curl -i` และ log ของเซิร์ฟเวอร์เพื่อยืนยัน



ตัวอย่างที่เห็นได้จริง

```
curl -i -X POST http://localhost:3001/api/books -H
'Content-Type: application/json' -d '{'
```

ลองคิดหรือสังเกต: ชี้ใน `app.js` ว่าตัวอ่าน JSON กับ error handler ลงทะเบียนอยู่ก่อนหรือหลัง route

เชื่อมกับแอป React + Vite: JSON จากฟอร์ม React ต้องอ่านได้ก่อนถึง route เพิ่มหนังสือ

อ่านหนังสือบทที่ 8 §8.8–8.9

ทบทวนเส้นทางและหลักฐานที่ต้องส่ง

เมื่อ React เรียก API คำขอผ่าน middleware ไปยัง route ตรวจข้อมูล แล้วจึงอ่านหรือเพิ่มหนังสือ คำตอบย้อนกลับไปให้หน้า React แสดงผล

ส่งโค้ดใน repository ส่วนตัว ตารางทำนายและผลจริงอย่างน้อย 10 คำขอ และใบงานที่อธิบายว่าแต่ละผลเกิดตรงไหนในโค้ด ดูหนังสือบทที่ 8 ประกอบทุกขั้น



ตัวอย่างที่เห็นได้จริง

หลักฐานหนึ่งแถว: `GET /api/books/abc` → คาดว่า `422` → ได้ `422` → `errors.id` ในคำตอบ

ลองคิดหรือสังเกต: ถ้าต้องเปลี่ยน store เป็นฐานข้อมูลในบทหน้า ส่วนใดของคำตอบ API ควรเหมือนเดิม?

เชื่อมกับแอป React + Vite: ผลจาก Express กลับมาที่ React แล้วเปลี่ยนสิ่งที่ผู้ใช้เห็น

อ่านหนังสือบทที่ 8 §8.10